

Programación Lógica

Fundamentos teóricos

LPO – Sintaxis

Cláusula

Una *cláusula* es una fórmula cerrada con la siguiente forma:

$$\forall x_1 \dots \forall x_k (L_1 \vee \dots \vee L_p)$$

en donde cada L_i es un literal y $x_1 \dots x_k$ son todas las variables que ocurren en $L_1 \vee \dots \vee L_p$.

LPO – Sintaxis

Cláusula

Vamos a utilizar las siguientes notaciones para cláusulas:

$$\forall x_1 \dots \forall x_k (A_1 \vee \dots \vee A_p \vee \neg B_1 \vee \dots \vee \neg B_q)$$

(equivalencia or/implica + de morgan)

$$(A_1 \vee \dots \vee A_p \leftarrow B_1 \wedge \dots \wedge B_q)$$

$$(A_1, \dots, A_p \leftarrow B_1, \dots, B_q)$$

$$\{A_1, \dots, A_p, \neg B_1, \dots, \neg B_q\}$$

En las últimas se asume la clausura universal.

LPO – Sintaxis

Resumen

- Cláusulas $(A_1 \vee \dots \vee A_p \leftarrow B_1 \wedge \dots \wedge B_q)$
 - Cláusulas de Horn
 - Cláusulas definidas
 - reglas: $(A_1 \leftarrow B_1 \wedge \dots \wedge B_q)$
 - hechos: $(A_1 \leftarrow)$
 - Objetivos definidos $(\leftarrow B_1 \wedge \dots \wedge B_q)$

(Recordar: Los A_i y B_i son literales. Asumimos la clausura universal.)

Semántica

Modelos

Def.- Sea F una fórmula cerrada e I una interpretación, I es un **modelo** para F si F es verdadera según I (I **satisface** F).

Def.- Sea S un conjunto de fórmulas cerradas, I es un modelo para S si satisface todas las fórmulas de S .

Semántica

Def. Sea S un conjunto de fórmulas cerradas

- S es **satisfactible** si existe al menos un modelo para S .
- S es **válido** (lógicamente válido) si toda interpretación es modelo.
- S es **insatisfactible** si ninguna interpretación es modelo.

Semántica

Teo. Sea S un conjunto de flas. cerradas y F una fla. cerrada sobre L de primer orden:

$S \models F$ sii $S \cup \{\neg F\}$ es insatisfactible.

Semántica

Teo. Sea S un conjunto de flas. cerradas y F una fla. cerrada sobre L de primer orden:

$S \models F$ sii $S \cup \{\neg F\}$ es insatisfactible.

Prueba $\Rightarrow$

Sea I una interpretación cualquiera.

- Si I es modelo de S , como $S \models F$, I es modelo de F . Entonces I no satisface $\neg F$, por lo que $S \cup \{\neg F\}$ es insatisfactible.
- Si I no es modelo de S , no satisface al menos una cláusula de S , por lo que tampoco puede ser modelo de $S \cup \{\neg F\}$.

Prueba $\Leftarrow$

Sea I un modelo de S . Como $S \cup \{\neg F\}$ es insatisfactible, I no es modelo de $\neg F$. Entonces I es modelo de F . Entonces todo modelo de S es modelo de F . Por definición de consecuencia lógica $S \models F$.

Pruebas en prolog

Para probar que una consulta C es consecuencia lógica de un programa lógico P basta con probar que $P \cup \{\neg C\}$ es insatisfactible.

A este tipo de demostración se le llama **prueba por refutación**: se niega lo que se quiere probar, llegando a un absurdo.

Debemos buscar una forma automática de resolver este problema, para esto se introducen los **métodos de resolución**.

Resolución

Juan estudia francés o inglés.

Juan no estudia inglés.

Se deduce:

Juan estudia francés.

$$\{P \vee Q, \neg P\} \Rightarrow Q$$

$$\{\neg \exists x P(x) \vee Q(a), Q(b) \vee \exists x P(x)\} \Rightarrow Q(a) \vee Q(b)$$

Resolución

El esquema de derivación que se utiliza se llama resolución

$$\frac{\alpha \vee \beta, \quad \gamma \vee \neg\beta}{\alpha \vee \gamma}$$

La propuesta de esta regla es de Robinson (1965) y está asociada a sistemas de demostración automática.

Esta regla es aplicable para α , β y γ fórmulas cualesquiera de 1er orden. Es inmediato ver que la regla es **correcta**. En efecto, si ambas premisas son verdaderas en una interpretación I , una al menos de las fórmulas α o γ deben ser verdaderas en I . Por lo tanto, $\alpha \vee \gamma$ es verdadera en I . Se cumple entonces que $\alpha \vee \gamma$ es consecuencia lógica de las premisas.

Resolución para cláusulas proposicionales

En un paso de resolución entre dos cláusulas se elimina **un par de literales complementarios** entre dos cláusulas. La cláusula resultante (la conclusión) es la disyunción de los resultantes literales.

Sean

$C1: L1 \vee L2 \vee \dots \vee L_n$

$C2: M1 \vee M2 \vee \dots \vee M_h$

y $L_i = \neg M_j$

$C3: L1 \vee L2 \vee \dots \vee L_{i-1} \vee L_{i+1} \vee \dots \vee L_n \vee M1 \vee \dots \vee M_{j-1} \vee M_{j+1} \vee \dots \vee M_h$

Se obtiene por resolución de $C1$ y $C2$.

Decimos que $C3$ es una resolvente de $C1$ y $C2$.

Resolución para cláusulas proposicionales

Ejemplo:

Deducir P a partir de $\{\{P, \neg Q, R\}, \{Q, S\}, \{\neg S\}, \{\neg R\}\}$

Resolución para cláusulas proposicionales

La cláusula vacía

Si nuestro conjunto de cláusulas es un par de literales complementarios en un paso de resolución llegamos a la cláusula vacía. Por ejemplo, resolviendo $\neg P$ y P obtenemos como resultado una cláusula sin literales.

La cláusula vacía (se escribe $\square$) no tiene modelos, puesto que no tiene literales que puedan hacerse verdaderos para una interpretación.

Como la resolución mantiene los modelos (la resolvente es consecuencia lógica de las dos cláusulas que se resuelven), si de un conjunto de cláusulas se obtiene la cláusula vacía por aplicación de uno o más pasos de resolución, podemos deducir que el conjunto de cláusulas no tiene ningún modelo. O sea, es insatisfactible.

Resolución para cláusulas de 1er orden

Necesitamos algunas definiciones previas:

- sustitución
- unificación
- unificador más general

Sustituciones

Si tenemos variables libres podemos sustituirlas por términos cualesquiera.

Definición:

Una sustitución θ es un conjunto finito de la forma $\{v_1/t_1, \dots, v_n/t_n\}$ en donde, para $1 \leq i \leq n$:

- v_i es una variable
- t_i es un término distinto de v_i
- $v_i \neq v_j$ si $i \neq j$

Ejemplos

¿Son sustituciones los siguientes conjuntos?

$\{x/1, y/f(h,x)\}$

$\{x/f(y), z/z\}$

$\{x/1, y/f(h,x), x/2\}$

Instancia

Def. Instancia de una fórmula según una sustitución

Sea E una expresión (término o fórmula sin cuantificadores),
y $\theta = \{v_1/t_1, \dots, v_n/t_n\}$ una sustitución :

$E\theta$, **instancia** de E por θ , es el resultado de la aplicación **simultánea** del reemplazo de cada variable v_i por el término t_i en E .

La noción de instancia se extiende a **conjuntos** de fórmulas:

Si $S = \{E_1, \dots, E_n\}$ es un conjunto de expresiones y θ una sustitución,
 $S\theta$, **instancia del conjunto** S por θ , es el conjunto $\{E_1\theta, \dots, E_n\theta\}$

Ejemplo

Si $E = p(x, y, f(b, x, z))$ y $\theta = \{x/a, y/x, z/f(z)\}$

$E\theta = ?$

Composición de sustituciones

Def. Composición de sustituciones

Sean $\sigma = \{v_1/s_1, \dots, v_n/s_n\}$ y $\theta = \{u_1/t_1, \dots, u_m/t_m\}$ dos sustituciones :
la **composición $\sigma \circ \theta$ de σ y θ** es la sustitución que resulta de

$\{v_1/s_1\theta, \dots, v_n/s_n\theta, u_1/t_1, \dots, u_m/t_m\}$

luego de eliminar:

- los pares $v_i/s_i\theta$ t.q. $v_i = s_i\theta$
- los pares u_j/t_j tales que $u_j \in \{v_1, \dots, v_n\}$

Ejemplo:

$\theta_1 = \{x|a, y|f(z), z|y\}$ y $\theta_2 = \{x|b, y|z, z|g(x)\}$ sustituciones.

$\theta_1 \circ \theta_2 = ?$

Unificación

Def. Unificador

Sea S un conjunto de expresiones.

Una sustitución θ es un unificador de S si $S\theta$ es un conjunto con un único elemento (*singleton*).

(Dicho de otro modo:

θ es unificador de S si para todas $E_i, E_j \in S$, $E_i\theta \equiv E_j\theta$)

Ejemplo:

$S = \{p(x, y, z), p(a, f(x), z)\}$

$\theta = \{x/a, y/f(a), z/b\}$ es un unificador, $S\theta = ?$

Unificador más general

Def. Unificador más general (m.g.u.)

θ es un mgu para $E1$ y $E2$ si:

- a. θ es un unificador para $E1$ y $E2$ y
- b. si σ es un unificador para $E1$ y $E2$, existe una sustitución γ t.q. $\theta \circ \gamma = \sigma$

El mgu de dos expresiones no tiene por qué ser único, p.ej., tanto $\theta1 = \{x/y\}$ como $\theta2 = \{y/x\}$ son mgus para $p(x)$ y $p(y)$.

Algoritmo de unificación

Entrada: E1 y E2

Salida: μ , mgu de las expresiones, o *no unifican* si E1 y E2 no son unificables.

Inicialización: PUSH E1 = E2; $\mu = \varepsilon$;

Mientras el stack no se vacíe

POP X=Y

En caso de que

1. X es una variable que no ocurre en Y:

$\mu = \mu \circ \{X/Y\}$;

sustituir Y por X en el stack, o sea, stack = stack $\circ \{X/Y\}$;

2. Y es una variable que no ocurre en X:

$\mu = \mu \circ \{Y/X\}$; stack = stack $\circ \{Y/X\}$;

3. X e Y son el mismo término: skip

4. X es $f(X_1, \dots, X_n)$, Y es $f(Y_1, \dots, Y_n)$:

PUSH $X_i = Y_i$; $i = 1 :: n$.

5. en otro caso: retornar *no unifican*.

Fin mientras

Retornar μ